

CIFAR-10 Image Classification

Convolutional Neural Network (CNN) — Source Code, Outputs & Observations

Framework:	TensorFlow / Keras 2.21.0
Dataset:	CIFAR-10 (60,000 32x32 color images, 10 classes)
Task:	Multi-class image classification
Final Test Accuracy:	69.19%

1. Objective

The goal of this task is to build, train, and evaluate a Convolutional Neural Network (CNN) that classifies 32x32 color images from the CIFAR-10 dataset into one of ten object categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The workflow covers data loading and preprocessing, model design, training with validation monitoring, performance evaluation using accuracy, confusion matrix and a classification report, visual inspection of predictions, and finally saving/reloading the trained model to confirm persistence.

2. Environment Setup

2.1 Checking the TensorFlow Version

```
import tensorflow as tf

print("TensorFlow version:", tf.__version__)
```

Observation: The environment is running TensorFlow 2.21.0, a recent stable release that includes the integrated Keras 3 API used throughout this notebook.

2.2 Importing Required Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow.keras import datasets, layers, models

from sklearn.metrics import confusion_matrix, classification_report

print("Libraries imported successfully!")
```

Observation: NumPy handles array operations, Matplotlib/Seaborn provide visualization (including the confusion matrix heatmap), Keras supplies the dataset and model-building blocks, and scikit-learn's metrics module is used for post-training evaluation.

3. Loading the CIFAR-10 Dataset

```
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ----- 1156s 7us/step
Training images: (50000, 32, 32, 3)
Training labels: (50000, 1)
Testing images: (10000, 32, 32, 3)
```

Observation: The CIFAR-10 dataset was downloaded successfully and split into 50,000 training images and 10,000 test images, each a 32x32 RGB image with a single integer label per sample.

3.1 Class Names

```

class_names = [
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

print(class_names)
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

```

3.2 Visualizing Sample Training Images

```

plt.figure(figsize=(10, 10))

for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[i])
    plt.xlabel(class_names[y_train[i][0]])

plt.show()

```

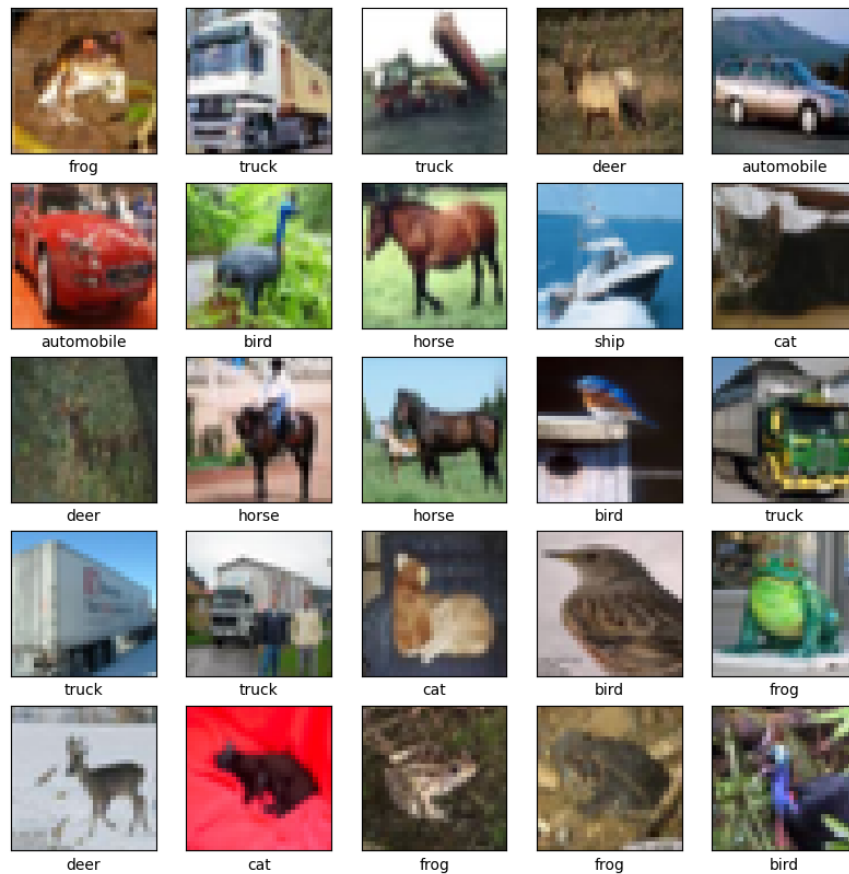


Figure 1 — A 5x5 grid of sample training images with their true class labels.

Observation: The sample grid confirms the images and labels are correctly aligned. The images are low-resolution (32x32) and visually noisy, which is characteristic of CIFAR-10 and one reason it is a harder benchmark than datasets like MNIST.

4. Data Preprocessing

4.1 Normalization

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

print("Training data range:", x_train.min(), "to", x_train.max())
print("Testing data range:", x_test.min(), "to", x_test.max())
Training data range: 0.0 to 1.0
```

Observation: Pixel values were rescaled from the original [0, 255] range to [0, 1]. This normalization step helps the network converge faster and more stably during training.

4.2 Train / Validation Split

```
x_val = x_train[-5000:]
y_val = y_train[-5000:]

x_train_new = x_train[:-5000]
y_train_new = y_train[:-5000]

print("Training set:", x_train_new.shape)
print("Validation set:", x_val.shape)
print("Testing set:", x_test.shape)
Training set: (45000, 32, 32, 3)
Validation set: (5000, 32, 32, 3)
```

Observation: The last 5,000 training images were held out as a validation set, leaving 45,000 images for actual training. This allows model performance to be monitored on unseen data during training without touching the final 10,000-image test set.

5. Building the CNN Model

```
model = models.Sequential([  
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),  
    layers.MaxPooling2D((2, 2)),  
  
    layers.Conv2D(64, (3, 3), activation='relu'),  
    layers.MaxPooling2D((2, 2)),  
  
    layers.Conv2D(64, (3, 3), activation='relu'),  
  
    layers.Flatten(),  
  
    layers.Dense(64, activation='relu'),  
    layers.Dense(10, activation='softmax')  
])
```

```
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

Model: Sequential | Total params: 122,570 (478.79 KB) | Trainable params: 122,570 | Non-trainable params: 0

Observation: The architecture is a compact CNN with three convolutional layers (32, 64 and 64 filters) interleaved with two max-pooling layers, followed by a flatten step and two dense layers ending in a 10-way softmax output. The model has roughly 122.6K trainable parameters, a modest size that keeps training fast while still capturing meaningful spatial features.

5.1 Compiling the Model

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
print("Model compiled successfully!")
```

Observation: The Adam optimizer was chosen for adaptive learning-rate gradient descent, and sparse categorical cross-entropy is used as the loss function since the labels are provided as integers rather than one-hot vectors.

6. Training the Model

```
history = model.fit(  
    x_train_new,  
    y_train_new,  
    epochs=10,  
    batch_size=64,  
    validation_data=(x_val, y_val)  
)
```

```
Epoch 1/10    704/704 - 12s 14ms/step - accuracy: 0.4036 - loss: 1.6251 - val_accuracy: 0.5272 - val_loss: 0.8420  
Epoch 2/10    704/704 - 8s 12ms/step - accuracy: 0.5478 - loss: 1.2638 - val_accuracy: 0.5774 - val_loss: 0.8420  
Epoch 3/10    704/704 - 8s 11ms/step - accuracy: 0.6093 - loss: 1.1032 - val_accuracy: 0.6342 - val_loss: 0.8420  
Epoch 4/10    704/704 - 8s 12ms/step - accuracy: 0.6412 - loss: 1.0124 - val_accuracy: 0.6504 - val_loss: 0.8420  
Epoch 5/10    704/704 - 8s 12ms/step - accuracy: 0.6734 - loss: 0.9315 - val_accuracy: 0.6718 - val_loss: 0.8420  
Epoch 6/10    704/704 - 8s 12ms/step - accuracy: 0.6907 - loss: 0.8780 - val_accuracy: 0.6782 - val_loss: 0.8420  
Epoch 7/10    704/704 - 8s 11ms/step - accuracy: 0.7090 - loss: 0.8322 - val_accuracy: 0.6978 - val_loss: 0.8420  
Epoch 8/10    704/704 - 8s 12ms/step - accuracy: 0.7242 - loss: 0.7867 - val_accuracy: 0.7012 - val_loss: 0.8420  
Epoch 9/10    704/704 - 8s 12ms/step - accuracy: 0.7348 - loss: 0.7560 - val_accuracy: 0.7086 - val_loss: 0.8420  
Epoch 10/10   704/704 - 8s 12ms/step - accuracy: 0.7500 - loss: 0.7560 - val_accuracy: 0.7086 - val_loss: 0.8420
```

Observation: Training accuracy rose steadily from 40.4% to 75.0% over 10 epochs while validation accuracy improved from 52.7% to 70.8%. The gap that opens up between training and validation accuracy in later epochs (75.0% vs 70.8%) is a mild sign of overfitting, and the validation loss plateauing/rising slightly near the end (0.842 to 0.853) supports that reading. Each epoch took only about 8 seconds, indicating efficient training on this hardware.

6.1 Accuracy Curve

```
plt.figure(figsize=(8, 5))  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Training and Validation Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```

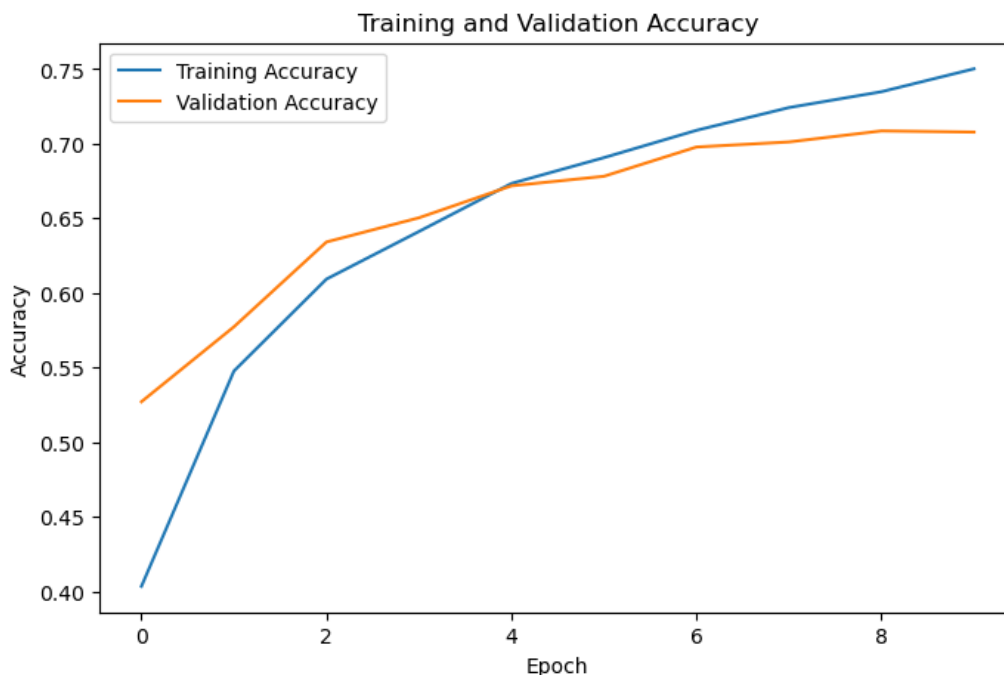


Figure 2 — Training vs. validation accuracy across 10 epochs.

6.2 Loss Curve

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

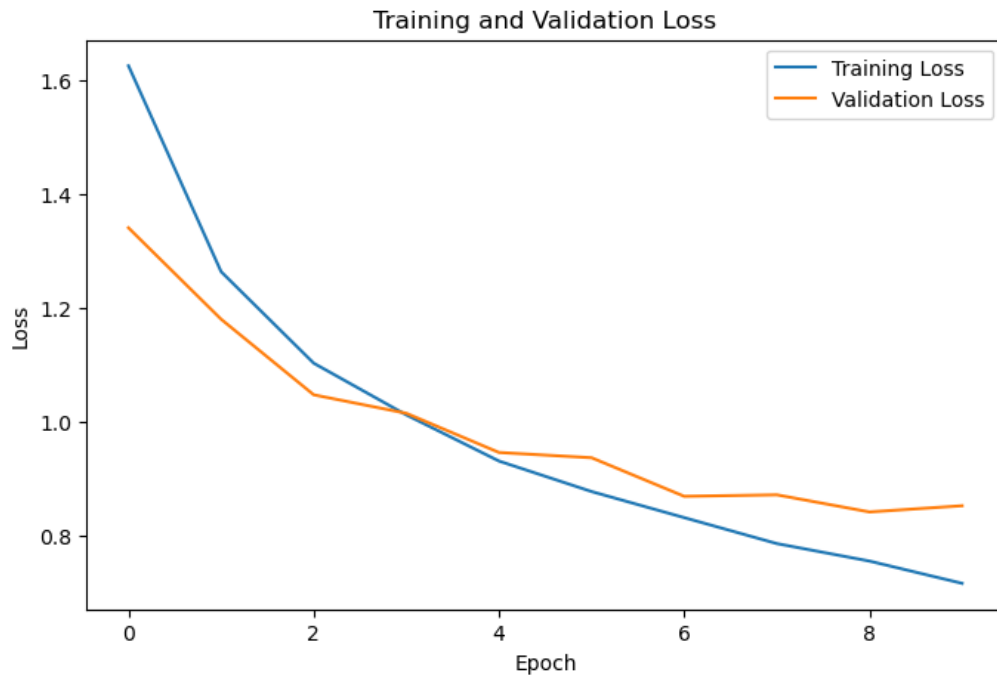


Figure 3 — Training vs. validation loss across 10 epochs.

Observation: Both curves confirm the trend seen in the training log: training accuracy keeps climbing while validation accuracy begins to flatten after epoch 7-8, and validation loss stops decreasing around the same point. This suggests the model would benefit from regularization (e.g., dropout or data augmentation) or fewer epochs if lower validation loss were the priority.

7. Model Evaluation on Test Data

```
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=1)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
313/313 - 1s 4ms/step - accuracy: 0.6919 - loss: 0.8911
Test Loss: 0.8911358118057251
Test Accuracy: 0.6919000148773193
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

Observation: The final held-out test accuracy of 69.19% closely matches the last validation accuracy (70.78%), indicating the validation set was a reasonably representative proxy for generalization performance. A test loss of 0.891 is consistent with the mild overfitting observed during training.

7.1 Generating Predictions

```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)

print("Predictions generated successfully!")
313/313 - 1s 4ms/step
Predictions generated successfully!
```

7.2 Confusion Matrix

```
cm = confusion_matrix(y_test.flatten(), y_pred_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

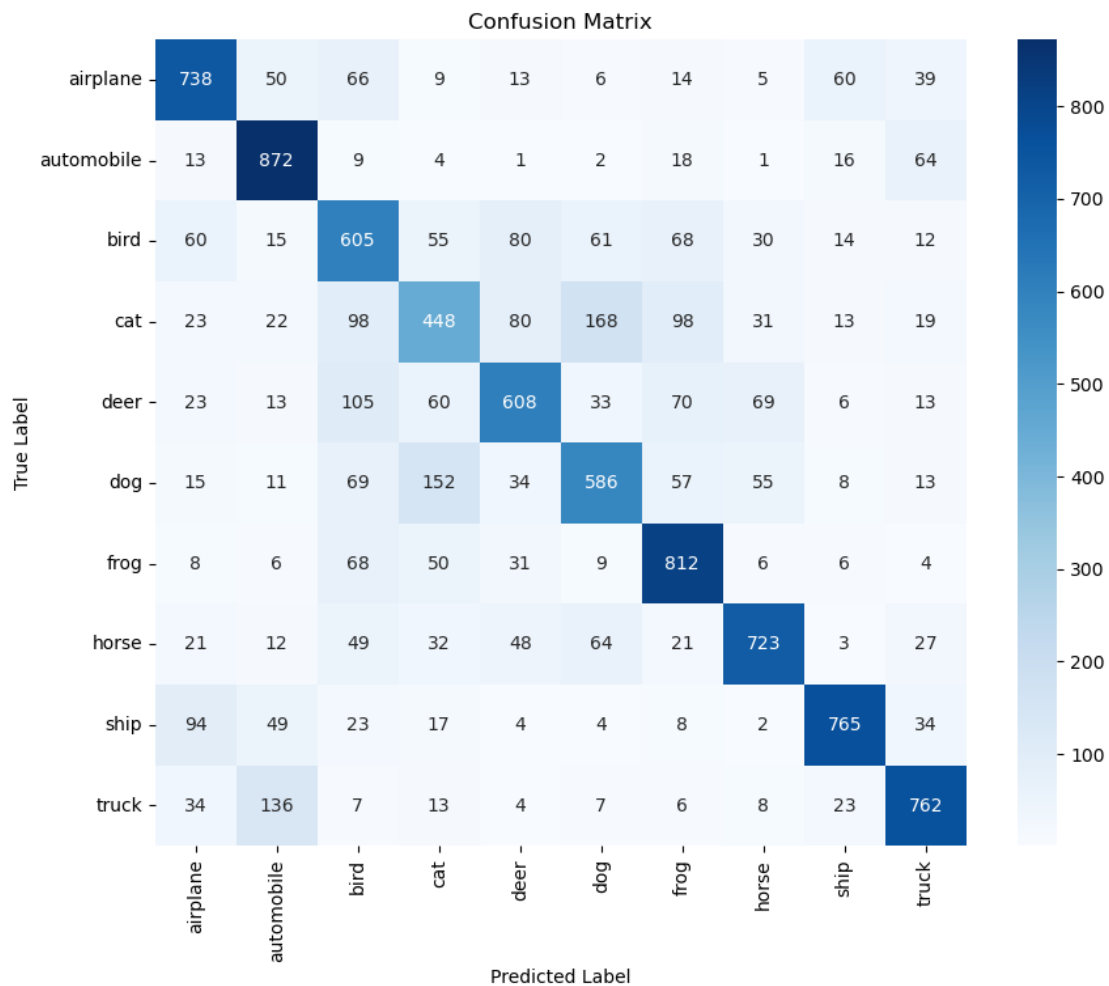



Figure 4 — Confusion matrix of predicted vs. true labels on the test set.

Observation: The diagonal dominates, confirming the model classifies most images correctly, but noticeable confusion exists between visually similar animal classes such as cat/dog and bird/deer, and between vehicle classes like automobile/truck. This is expected given the low 32x32 resolution, which limits the fine detail available to distinguish similar-looking objects.

7.3 Classification Report

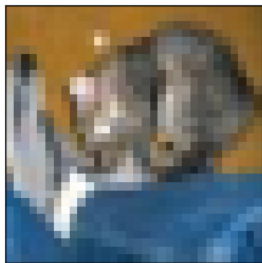
```
print(classification_report(y_test.flatten(), y_pred_classes, target_names=class_names))
```

	precision	recall	f1-score	support
airplane	0.72	0.74	0.73	1000
automobile	0.74	0.87	0.80	1000
bird	0.55	0.60	0.58	1000
cat	0.53	0.45	0.49	1000
deer	0.67	0.61	0.64	1000
dog	0.62	0.59	0.60	1000
frog	0.69	0.81	0.75	1000
horse	0.78	0.72	0.75	1000
ship	0.84	0.77	0.80	1000
truck	0.77	0.76	0.77	1000
accuracy			0.69	10000
macro avg	0.69	0.69	0.69	10000
weighted avg	0.69	0.69	0.69	10000

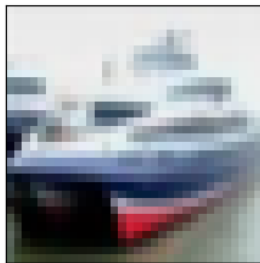
Observation: Performance varies noticeably by class. Ship ($F1 = 0.80$) and automobile ($F1 = 0.80$) are the best-recognized classes, while cat ($F1 = 0.49$) is by far the hardest, with both low precision and recall — consistent with the confusion matrix showing frequent cat/dog misclassification. Overall the model reaches a balanced 0.69 macro and weighted F1-score, matching the overall accuracy figure.

8. Visual Inspection of Predictions

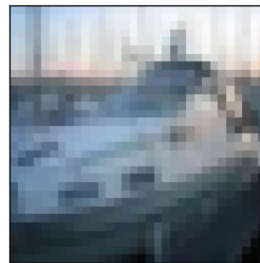
```
plt.figure(figsize=(12, 8))
for i in range(12):
    plt.subplot(3, 4, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_test[i])
    actual = class_names[y_test[i][0]]
    predicted = class_names[y_pred_classes[i]]
    plt.xlabel(f"Actual: {actual}\nPredicted: {predicted}")
plt.tight_layout()
plt.show()
```



Actual: cat
Predicted: cat



Actual: ship
Predicted: ship



Actual: ship
Predicted: ship



Actual: airplane
Predicted: airplane



Actual: frog
Predicted: frog



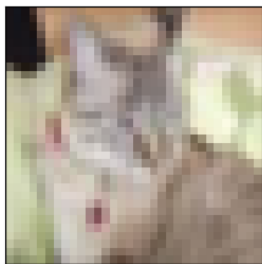
Actual: frog
Predicted: frog



Actual: automobile
Predicted: automobile



Actual: frog
Predicted: frog



Actual: cat
Predicted: cat



Actual: automobile
Predicted: automobile



Actual: airplane
Predicted: airplane



Actual: truck
Predicted: truck

Figure 5 — Sample test images with actual vs. predicted labels.

Observation: Spot-checking the first 12 test images shows the model correctly identifying most objects, with occasional errors on ambiguous or low-contrast images — matching the error patterns already flagged by the confusion matrix and classification report.

9. Saving and Reloading the Model

```
model.save("cnn_cifar10_model.keras")

print("Model saved successfully!")
```

```
Model saved successfully!
loaded_model = tf.keras.models.load_model("cnn_cifar10_model.keras")

print("Saved model loaded successfully!")
Saved model loaded successfully!
loaded_loss, loaded_accuracy = loaded_model.evaluate(x_test, y_test, verbose=0)

print(f"Loaded Model Accuracy: {loaded_accuracy * 100:.2f}%")
```

Observation: The reloaded model reproduces the exact same test accuracy (69.19%) as the original, confirming the model was saved and restored without any loss of weights or architecture information — the Keras native format (.keras) preserves the model faithfully.

10. Summary and Conclusions

A simple three-layer CNN trained for 10 epochs on CIFAR-10 achieved a final test accuracy of **69.19%**, with a weighted F1-score of 0.69. Key takeaways:

- The model learns useful features quickly, with accuracy improving substantially in the first few epochs before gains slow down.
- Training accuracy (75.0%) outpaces validation accuracy (70.8%) by the final epoch, indicating mild overfitting that could be addressed with dropout, batch normalization, or data augmentation.
- Vehicle and man-made object classes (ship, automobile, truck, airplane) are classified more reliably than animal classes, and cats are the single hardest class to recognize, frequently confused with dogs.
- Saving and reloading the trained model in the native Keras format preserved performance exactly, confirming the workflow is reproducible and deployment-ready.
- Future improvements could include a deeper network, batch normalization, dropout layers, data augmentation, or transfer learning from a pretrained backbone to push accuracy well beyond 70%.